Generics: الشرح النظري

تخيّل أن عندك **مستودعاً** تضع فيه الأشياء داخل صناديق. في هذا الملف سنمشي معك قصة هذا المستودع خطوة خطوة: نبدأ بمشكلة بسيطة، ثم نجرّب حلاً قديماً، ثم نرى لماذا لم يعد كافياً، وفي النهاية نصل إلى الحل الجميل: **الـ Generics**. اقرأ بالترتيب ولا تقفز، فكل خطوة تمهّد للتي بعدها.

ماذا ستتعلم؟ لماذا نستعمل Generics ومعنى Type Safety، ثم الصف العام والدالة العامة والواجهة العامة، ثم قيود Generics الناتجة عن Type Erasure.

1 المشكلة: نكتب صفاً لكل نوع!

تريد أن تصنع **صندوقاً** يحفظ شيئاً واحداً. في البداية تحتاج صندوقاً للنصوص، فتكتب له صفاً. بعد قليل تحتاج صندوقاً للأعداد الصحيحة، فتكتب صفاً ثانياً. ثم صندوقاً للأعداد العشرية، فتكتب صفاً ثالثاً. هكذا:

StringBox

class StringBox {
String value;
}

IntBox

class IntBox {
int value;
}

DoubleBox

class DoubleBox {
double value;
}

ثلاثة صفوف متطابقة تماماً، والفرق الوحيد: نوع الحقل (بالأحمر)

الحل البسيط

```
class StringBox {
    String value;
}
class IntBox {
    int value;
}
class DoubleBox {
    double value;
}
```

نوع الحقل: نص

نوع الحقل: عدد صحيح

نوع الحقل: عدد عشري

انظر جيداً: الصفوف الثلاثة **متطابقة**، والاختلاف الوحيد كلمة واحدة هي نوع الحقل. تخيّل الآن أنك تحتاج صندوقاً للطلاب، ثم للمنتجات، ثم للفواتير... ستنسخ الصف مرة بعد مرة وتغيّر كلمة واحدة! هذا تعب، وقد تنسى فتخطئ.

السؤال الذي في بالك الآن: ألا يوجد صندوق **واحد** يصلح لكل الأنواع؟ نعم يوجد حل قديم، فلنجرّبه أولاً.

2 الحل القديم: Object

هذه فكرة ذكية: في جافا **كل الأشياء** (النصوص والأعداد والطلاب...) هي في النهاية `Object`. فإذا صنعنا صندوقاً يخزّن `Object` فسيقبل أي شيء! جرّب:

ObjectBox.java

```java
package generics;

public class ObjectBox {
    private Object value;

    public ObjectBox(Object value) {
        this.value = value;
    }

    public Object get() {
        return value;
    }

    public void set(Object value) {
        this.value = value;
    }
}
```

يقبل أي شيء

يرجع Object فقط

CrashDemo.java

```java
package generics;

public class CrashDemo {
    public static void main(String[] args) {
        ObjectBox box = new ObjectBox("Java");
        System.out.println("Stored: " + box.get());
        Integer number = (Integer) box.get();
        System.out.println(number + 1);
    }
}
```

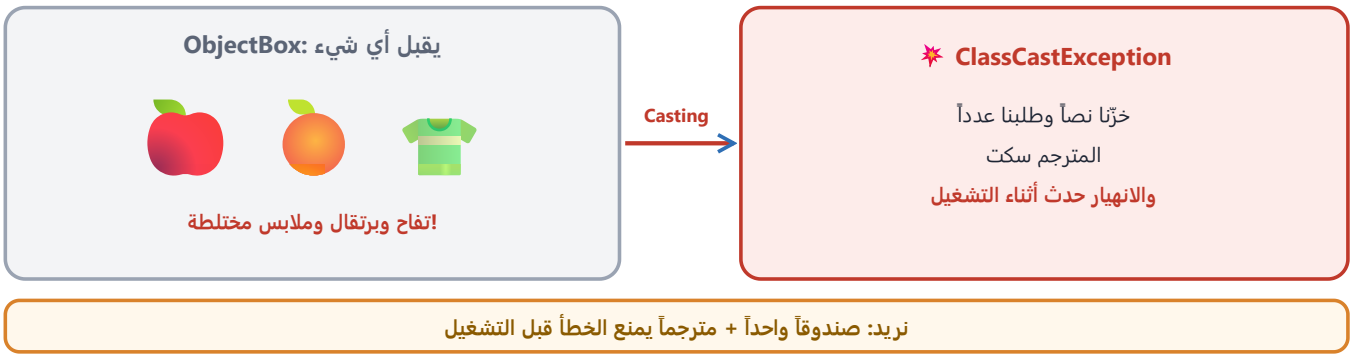
خزّنا نصاً: مقبول لأن النص هو Object

Casting: نجبر المترجم أن يعتبره عدداً

الصندوق يعمل، وخزّنا فيه نصاً بلا مشكلة. لكن انظر ماذا فعلنا في السطر الأخير: خزّنا **نصاً** ثم طلبنا منه **عدداً**! هل سيعترض المترجم؟ لا، لقد وافق على الكود. شغّله لترى ماذا يحدث:

الناتج الفعلي: الانهيار وقت التشغيل

```
Stored: Java
Exception in thread "main" java.lang.ClassCastException: java.lang.String cannot be cast to java.lang.Integer
    at generics.CrashDemo.main(CrashDemo.java:10)
```



لماذا هذا الحل خطير؟

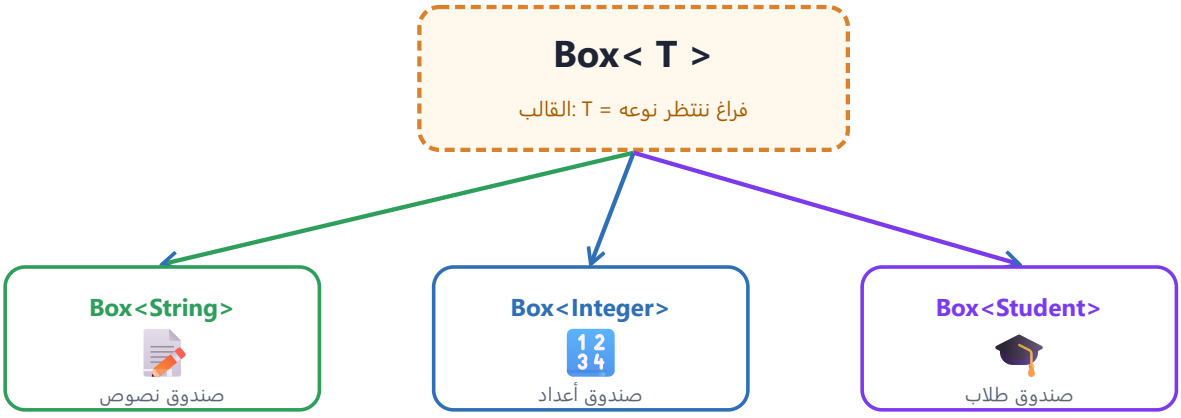
المترجم **لا يعرف** ما بداخل الصندوق. عندما كتبنا `(Integer)` قلنا له: "ثق بي، هذا عدد". فصدّقنا ومرّ الكود. لكن عند التشغيل اكتشف جافا أن الذي بالداخل نص، فتوقف البرنامج برسالة `ClassCastException` .

والخطر الأكبر: الانهيار يحدث **أثناء التشغيل**، أي ربما بعد أن يستعمل الزبون برنامجك! وليس وأنت تكتب.

إذن نحتاج شيئاً يجمع أمرين معاً: **صندوقاً واحداً لكل الأنواع**، و**مترجماً يمنع الخطأ قبل التشغيل**. وهذا بالضبط ما تعطيه الـ **Generics**.

3 الفكرة: معامل النوع (Type Parameter)

الفكرة بسيطة جداً. فكّر في استمارة فيها فراغ: "الاسم: _____". لا تعرف ما سيُكتب فيه الآن، لكن الاستمارة جاهزة. سنفعل الشيء نفسه مع الصندوق: نترك **فراغاً للنوع**، ونملؤه لاحقاً عندما نصنع الصندوق. هذا الفراغ نسميه `T` ، ونكتب النوع بين `< >` :



الكلمة	معناها ببساطة
`< >`	نكتب بينهما **نوع** ما بداخل الصندوق
`T`	اسم مؤقت يحجز **مكان النوع**، ويسمى معامل النوع (Type Parameter). مثل فراغ في ورقة اسمه T
`Box<String>`	ملأنا الفراغ بـ `String` : صار صندوق نصوص
`Box<Integer>`	ملأناه بـ `Integer` : صار صندوق أعداد

تشبيه: الصف العام مثل **قالب كعك**: القالب واحد، ونصب فيه كل مرة عجينة مختلفة. نكتب `Box<String>` فنصبّ في القالب نوع `String` ، ونكتب `Box<Integer>` فنصبّ نوعاً آخر.

هل رأيت هذا من قبل؟ نعم! `ArrayList<String>` هي قائمة نصوص فقط، و `ArrayList<Student>` قائمة طلاب فقط. الصف `ArrayList` واحد، ونوع ما يحمله يُكتب بين `<` `>` . أنت تستعمل Generics منذ أول المقرر.

4 نكتب الصف العام خطوة بخطوة

Box.java

```
package generics;

public class Box<T> {
    private T value;

    public Box(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }

    public void set(T value) {
        this.value = value;
    }
}
```

الخطوة 1: نكتب بعد اسم الصف

الخطوة 2: T بدل النوع

T في وسيط البنّاء

T نوع إرجاع

T في وسيط التابع

كيف نكتب الصندوق العام؟ ببساطة: نأخذ `ObjectBox` ونفعل شيئين فقط:

- **أولاً:** نكتب `<T>` بعد اسم الصف، لنقول "هذا الصف فيه فراغ اسمه T".
- **ثانياً:** في كل مكان كان فيه `Object` نكتب `T` .

هذا كل شيء! داخل الصف نتعامل مع `T` كأنه نوع عادي (حقل، ووسيط، ونوع إرجاع). ولا يهمنا ما هو الآن، فسيتحدد عندما ننشئ الصندوق.

لاحظ أن الكود مطابق لـ `ObjectBox` تماماً، مع استبدال `Object` بـ `T` .

ماذا يفعل المترجم عندما ننشئ غرضاً؟

ماذا يحدث عندما نصنع صندوقاً؟ عندما نكتب `Box<String>` يقوم المترجم بعمل بسيط: **يستبدل** `T` **بـ** `String` **في كل مكان**. فيرى الصف كأنك كتبته هكذا:

Box<String> ما يراه المترجم لـ

```
class Box {
    private String value;
    Box(String value) { ... }
    String get() { ... }
    void set(String value) { ... }
}
```

T أصبحت String

ترجع String مباشرة: بلا Casting

تقبل String فقط

انظر النتيجة: `get` ترجع `String` مباشرة دون `Casting`، و `set` تقبل `String` فقط. وإن حاولنا إدخال عدد، يعترض المترجم **فوراً**.

> الرمز `<>` في الطرف الأيمن يسمى **Diamond**، ومعناه: **استنتج النوع مما كتبته في الطرف الأيسر**. فلا نكرر `String` مرتين.

5 أمان الأنواع (Type Safety)

لنرى ماذا ربحنا. نصنع صندوق نصوص ونحاول أن نعبث به:

الاستعمال

```
Box<String> b = new Box<>("Java");
b.set("Swing");
b.set(5);
String s = b.get();
```

مقبول: نص

خطأ ترجمة: عدد في صندوق نصوص

بلا Casting: المترجم يعرف أنه String

خطأ أحمر وأنت تكتب (قبل التشغيل):

```
incompatible types: int cannot be converted to String
```

هذا هو الربح الكبير: السطر الثالث يعطي **خطأً أحمر وأنت تكتب**، قبل أن تشغّل شيئاً. يسمى هذا **أمان الأنواع (Type Safety)**: المترجم يمنعك من وضع نوع خاطئ، فلا يصل الخطأ أبداً إلى المستخدم.

	ObjectBox (قبل)	Box<T> (Generics)
وضع نوع خاطئ	مقبول (لا يعرف المترجم)	خطأ ترجمة فوراً
القراءة	نحتاج Casting	مباشرة بلا Casting
متى يظهر الخطأ؟	أثناء التشغيل (متأخر)	أثناء الكتابة (مبكر)
الاسم	ClassCastException	Compile Error

> لهذا نقول: الـ Generics **تنقل الخطأ من وقت التشغيل إلى وقت الترجمة**. وهذا أفضل بكثير، لأنك تكتشفه وأنت تكتب.

6 أكثر من معامل نوع: Pair

حسناً، الصندوق يحمل شيئاً واحداً. وماذا لو أردنا أن نحفظ **شيئين مختلفين** معاً، مثل اسم طالب (نص) وعمره (عدد)؟ نستعمل فراغين بدل فراغ واحد، ونفصل بينهما بفاصلة:

Pair.java (جزء)

```
class Pair<K, V> {
    private K key;
    private V value;
}

Pair<String, Integer> p;
```

معاملان: K , V

الأول من نوع K

الثاني من نوع V

K = String , V = Integer

عند الإنشاء نكتب نوعاً لكل معامل **بالترتيب**: `Pair<String, Integer>` تعني أن `K` هو `String` و `V` هو `Integer`.

ماذا تعني الحروف T و E و K و V؟

الحرف	اختصار	مثال
`T`	Type (نوع)	`Box<T>`
`E`	Element (عنصر)	عنصر داخل قائمة
`K` و `V`	Key (مفتاح) و Value (قيمة)	`Pair<K, V>`
`N`	Number (عدد)	نوع عددي

> هذه الأسماء **اتفاق بين المبرمجين** فقط، ولا شيء يمنعك من استعمال أي اسم آخر. لكن اتباع العُرف يجعل كودك مفهوماً للجميع.

7 لماذا لا نكتب int؟

قبل أن نكمل، تنبيه صغير يسبب أخطاء كثيرة للمبتدئين. جرّب أن تكتب `Box<int>`:

خطأ أحمر وأنت تكتب (قبل التشغيل):

```
unexpected type
required: reference
found:    int
```

السبب: الفراغ `T` لا يقبل إلا **صفاً**، و `int` ليس صفاً بل نوع أولي. والحل بسيط: لكل نوع أولي **صف مغلِّف** يقابله، فنكتب `Box<Integer>` بدل `Box<int>`:

الصفوف المغلِّفة (Wrapper Classes)

النوع الأولي	صفه المغلِّف
`int`	`Integer`
`double`	`Double`
`boolean`	`Boolean`
`char`	`Character`



والجميل أنك لا تتعب: جافا تحوّل **تلقائياً** بين الاثنين. وضع `5` داخل `Box<Integer>` يسمى **Autoboxing**، وقراءته إلى `int` يسمى **Unboxing**. اكتب كودك كالمعتاد.

8 الدوال العامة (Generic Methods)

الآن سؤال جديد: وماذا لو لم نكن نريد **صندوقاً** أصلاً، بل **عملاً واحداً** يجب أن ينجح مع أي نوع؟ مثلاً دالة تطبع عناصر مصفوفة: كود الطباعة نفسه سواء كانت أعداداً أو نصوصاً. هنا لا نحتاج صفاً عاماً، بل **دالة عامة**:

دالة عامة

```
static <T> void printArray(T[] arr) {
    for (T item : arr) {
        System.out.print(item + " ");
    }
}
```

نكتب قبل نوع الإرجاع مباشرة

T في الوسيط

T في متغير الحلقة

انظر أين وضعنا `<T>`: **قبل نوع الإرجاع مباشرة**. وبعدها استعملنا `T` داخل الدالة. وهذا الفراغ **خاص بالدالة وحدها**، لا علاقة له بالصف الذي هي فيه.

لو نسيت كتابة قبل نوع الإرجاع:

```
cannot find symbol
symbol:   class T
location: class E5
```

كيف نستدعيها؟

الاستدعاء

```
Integer[] nums = {1, 2, 3};
String[] names = {"Sara", "Omar"};

printArray(nums);
printArray(names);
```

المترجم يستنتج: T = Integer

T = String

والآن الجميل: لا نكتب النوع عند الاستدعاء! المترجم **يستنتجه** بنفسه من الوسيط، ويسمى هذا **Type Inference**. فعند `printArray(nums)` يرى أن `nums` من نوع `Integer[]` فيقرر أن `T` هو `Integer`.

	معامل الصف	معامل الدالة
أين نكتبه؟	بعد اسم الصف: `Box<T>`	قبل نوع الإرجاع: `<T> void f()`
متى يتحدد؟	عند **إنشاء الغرض**	عند **كل استدعاء**
النتيجة	الغرض كله بنوع واحد	الدالة نفسها تعمل مع أنواع مختلفة في استدعاءات مختلفة

9 الواجهة العامة (Generic Interface)

تعلمنا الصف العام والدالة العامة. بقي شيء ثالث: **الواجهة العامة**. تذكّر أن الواجهة **وعد** بتابع. فهذه الواجهة تعد بتابع **يحوّل من نوع A إلى نوع B**، دون أن تقرر ما هما النوعان:

Converter.java

```java
package generics;

public interface Converter<A, B> {
    B convert(A input);
}
```

A هو المدخل، و B هو المخرج

تستقبل A وترجع B

ومن ينفّذ الواجهة هو الذي يقرر النوعين. هنا قررنا: المدخل نص، والمخرج عدد (طول النص):

LengthConverter.java

```java
package generics;

public class LengthConverter implements Converter<String, Integer> {

    @Override
    public Integer convert(String s) {
        return s.length();
    }
}
```

A = String , B = Integer

يجب أن تطابق الأنواع المحددة

طول النص

داخل هذا الصف صارت `A` هي `String` و `B` هي `Integer`، ويجب أن تطابق الدالة هذه الأنواع. ويمكن أيضاً أن يبقى الصف عاماً ويمرّر المعاملات: `class X<A, B> implements Converter<A, B>`.

تشبيه: الواجهة العامة مثل **آلة تحويل**: تحمل لافتة "أعطني نوعاً وأعطيك نوعاً آخر"، وكل آلة (صف) تقرر ما هما النوعان.

10 Type Erasure: ماذا يحدث بعد الترجمة؟



سؤال أخير يفسّر قيوداً غريبة سنراها: ماذا يحدث للفراغ `T` بعد أن ينتهي المترجم؟ الجواب: **يمحوه!** بعد أن يفحص الأنواع ويطمئن، يحذف معاملات النوع ويبدّل `T` بـ `Object`. يسمى هذا **Type Erasure** أي **محو الأنواع**. والسبب أن الكود القديم قبل Generics يجب أن يبقى يعمل.

إذن: المهمة الحقيقية لـ Generics هي **الفحص أثناء الترجمة**. أما وقت التشغيل فلا فرق بين `Box<String>` و `Box<Integer>`. وهذه الحقيقة تفسر القيود التالية.

قيود Generics بسبب المحو

بما أن جافا لا تعرف `T` وقت التشغيل، تصبح بعض الأشياء **ممنوعة**:

الممنوع	الخطأ الفعلي من المترجم	لماذا؟
`new T()`	unexpected type required: class, found: type parameter T	`T` غير معروف وقت التشغيل، فلا يعرف جافا أي صف ينشئ
`new T[5]`	generic array creation	لنفس السبب
`x instanceof Box<String>`	illegal generic type for instanceof	`Box<String>` و `Box<Integer>` يصبحان `Box` عند التشغيل
`static T s;`	non-static type variable T cannot be referenced from a static context	الحقل `static` مشترك بين كل الأغراض، بينما لكل غرض `T` مختلف
`Box<int>`	unexpected type required: reference, found: int	المعامل يجب أن يكون صفاً (رأينا هذا)

11 أسئلة قد تخطر ببالك

❓ **فكّر: 1) لماذا لا نستعمل `Object` ونرتاح؟ ما الفرق؟**

`Object` يقبل **أي شيء**، فنضطر لكتابة `Casting` وقد نخطئ بلا أن يعلم المترجم، فينهار البرنامج أثناء التشغيل. مع Generics يمنعك المترجم **وأنت تكتب**.

❓ **فكّر: 2) ما هو `T` بالضبط؟ هل هو صف موجود؟**

لا، ليس صفاً. هو **اسم مؤقت** يحجز مكان النوع حتى نحدده. مثل الفراغ في ورقة: لا نعرف ما فيه حتى نملأه.

❓ **فكّر: 3) هل يجب أن أسمّيه `T` ؟**

لا. أي اسم يصلح. لكن العرف أن نكتب `T` للنوع، و `E` للعنصر، و `K` و `V` للمفتاح والقيمة، ليفهم الجميع كودك.

❓ **فكّر: 4) ما الفرق بين `Box<String>` و `Box<>` ؟**

`Box<String>` نكتبه على **الطرف الأيسر** لنقول نوع المتغير. و`Box<>`(Diamond) في الطرف الأيمن يعني: "استنتج النوع مما على اليسار".

❓ **فكّر: 5) لماذا `Box<int>` خطأ؟**

لأن المعامل يجب أن يكون **صفاً**، و `int` نوع أولي وليس صفاً. نكتب `Box<Integer>` وجافا تحوّل تلقائياً (Autoboxing وUnboxing).

❓ **فكّر: 6) هل يمكن أن أخزّن `int` في `Box<Integer>` مباشرة؟**

نعم: `new Box<>(25)` تعمل، لأن جافا تحوّل `25` إلى `Integer` تلقائياً.

❓ **فكّر: 7) لماذا نحتاج الدالة العامة؟ ألا يكفي الصف العام؟**

الصف العام يربط **الغرض كله** بنوع واحد. أما إذا كنت تريد **دالة واحدة** تعمل مع أنواع مختلفة (مثل طباعة أي مصفوفة)، فلا داعي لصف كامل، وتكفي دالة عامة.

❓ **فكّر: 8) لماذا لا نكتب النوع عند استدعاء الدالة العامة؟**

لأن المترجم **يستنتجه** من الوسيط (Type Inference). مثلاً `printArray(nums)` يرى أن `nums` من `Integer[]` فيجعل `T` هو `Integer` .

❓ **فكّر: 9) هل يمكن أن يكون للدالة معامل نوع مختلف عن معامل الصف؟**

نعم. معامل الدالة **خاص بها** ولا علاقة له بمعامل الصف، ويتحدد عند كل استدعاء.

❓ **فكّر: 10) ما الفرق بين الواجهة العادية والعامة؟**

الواجهة العامة (`Converter<A, B>`) تترك **أنواع** المدخل والمخرج مفتوحة، والصف الذي ينفّذها هو الذي يحددها.

❓ **فكّر: 11) ما فائدة معرفة Type Erasure؟**

تفسّر لك **لماذا** بعض الأشياء ممنوعة (مثل `new T()`). فجافا لا تعرف `T` وقت التشغيل، لأنها محته بعد الترجمة.

❓ **فكّر: 12) هل `Box<String>` و `Box<Integer>` صفان مختلفان وقت التشغيل؟**

لا. بعد المحو كلاهما `Box` واحد. الفحص يتم **أثناء الترجمة** فقط.

❓ **فكّر: 13) أين سأستعمل Generics في مشاريعي؟**

في كل مكان تقريباً: `ArrayList<Student>` و `HashMap<String, Integer>` و `Comparable<T>` ... كلها Generics. أنت تستعملها كل يوم.

12 الخلاصة

أريد...	أكتب	مثال
صفاً يحمل نوعاً يتحدد عند الإنشاء	`<T>` بعد اسم الصف	`class Box<T>`
صفاً يحمل نوعين	معاملان مفصولان بفاصلة	`class Pair<K, V>`
دالة تعمل مع أي نوع	`<T>` قبل نوع الإرجاع	`static <T> void f(T[] a)`
واجهة تترك الأنواع مفتوحة	`<A, B>` بعد اسم الواجهة	`interface Converter<A, B>`
أن أضع أعداداً	الصف المغلِّف	`Box<Integer>` وليس `Box<int>`

الفكرة في جملة: نكتب الصف **مرة واحدة** بفراغ اسمه `T` ، وعند الاستعمال **نملأ الفراغ** بالنوع الذي نريده، والمترجم **يحرس** أن لا يدخل غيره.